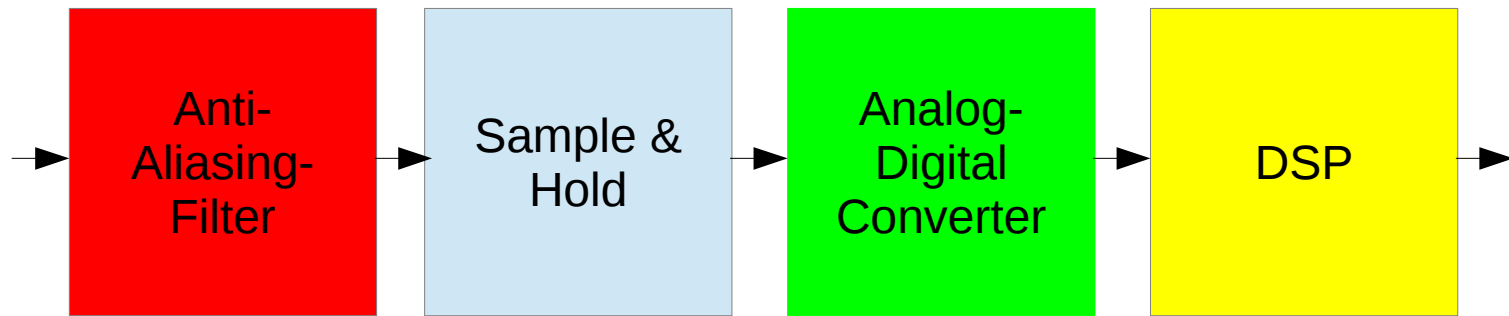


Digital Signal Processing

- Signed and unsigned **integer** numbers
CM4 supports 8 / 16 / 32 and 64 bit result formats
- **Fractional** numbers
CM4 supports 8 / 16 / 32 / 64 bit number formats
- **Floating point** numbers
CM4 offers hardware support
for 32 bit single precision floating point numbers

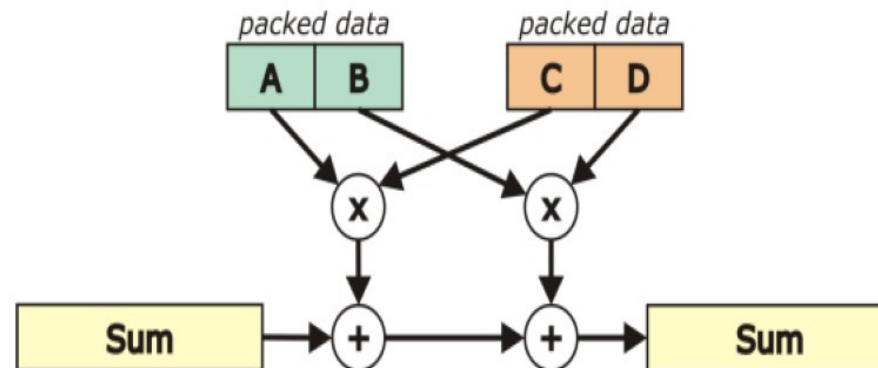
DSP Systems



Integer computation

Counting / arithmetics with integer numbers as usual

- Does not really match the needs of DSP algorithms
- Overflow in add / sub operations may occur
- Overflow in multiply operations is always problematic
- CM4 hardware supports saturating arithmetics
- CM4 hardware directly supports 8 / 16 / 32 bit data width
- Several parallel computing commands are available



Only one command "SMLALD"
16 bit input data A B C D
64 bit accumulator for Sum

ADD, ADC, SUB, SBC, and RSB

Add, Add with carry, Subtract, Subtract with carry, and Reverse Subtract.

Syntax

op{*S*}{*cond*} {*Rd*,} *Rn*, *Operand2*

op{*cond*} {*Rd*,} *Rn*, #*imm12* ; ADD and SUB only

where:

op Is one of:

ADD	Add.
ADC	Add with Carry.
SUB	Subtract.
SBC	Subtract with Carry.
RSB	Reverse Subtract.

S Is an optional suffix. If *S* is specified, the condition code flags are updated on the result of the operation,

cond Is an optional condition code,

Rd Specifies the destination register. If *Rd* is omitted, the destination register is *Rn*.

Rn Specifies the register holding the first operand.

Operand2 Is a flexible second operand.

imm12 Is any value in the range 0-4095.

Multiply and divide

Multiply	Multiply	MUL Rd, Rn, Rm	1
	Multiply accumulate	MLA Rd, Rn, Rm	1
	Multiply subtract	MLS Rd, Rn, Rm	1
	Long signed	SMULL RdLo, RdHi, Rn, Rm	1
	Long unsigned	UMULL RdLo, RdHi, Rn, Rm	1
	Long signed accumulate	SMLAL RdLo, RdHi, Rn, Rm	1
	Long unsigned accumulate	UMLAL RdLo, RdHi, Rn, Rm	1
Divide	Signed	SDIV Rd, Rn, Rm	2 to 12 ^a
	Unsigned	UDIV Rd, Rn, Rm	2 to 12 ^a

CM4 directly supports “long” 64-bit multiplication results !
Single cycle multiply, and 2 - max. 12 cycle division.

Saturating numbers

SSAT and USAT

Signed Saturate and Unsigned Saturate to any bit position, with optional shift before saturating.

Syntax

op{cond} Rd, #n, Rm {, shift #s}

where:

<i>op</i>	Is one of: SSAT Saturates a signed value to a signed range. USAT Saturates a signed value to an unsigned range.
<i>cond</i>	Is an optional condition code, see
<i>Rd</i>	Specifies the destination register.
<i>n</i>	Specifies the bit position to saturate to: • <i>n</i> ranges from 1 to 32 for SSAT • <i>n</i> ranges from 0 to 31 for USAT.
<i>Rm</i>	Specifies the register containing the value to saturate.
<i>shift #s</i>	Is an optional shift applied to <i>Rm</i> before saturating. It must be one of the following: ASR <i>#s</i> where <i>s</i> is in the range 1 to 31 LSL <i>#s</i> where <i>s</i> is in the range 0 to 31.

Saturating arithmetics

Mnemonic	Brief description
SSAT	Signed Saturate
SSAT16	Signed Saturate Halfword
USAT	Unsigned Saturate
USAT16	Unsigned Saturate Halfword
QADD	Saturating Add
QSUB	Saturating Subtract
QSUB16	Saturating Subtract 16
QASX	Saturating Add and Subtract with Exchange
QSAX	Saturating Subtract and Add with Exchange
QDADD	Saturating Double and Add
QDSUB	Saturating Double and Subtract
UQADD16	Unsigned Saturating Add 16
UQADD8	Unsigned Saturating Add 8
UQASX	Unsigned Saturating Add and Subtract with Exchange
UQSAX	Unsigned Saturating Subtract and Add with Exchange
UQSUB16	Unsigned Saturating Subtract 16
UQSUB8	Unsigned Saturating Subtract 8

All combinations of
signed / unsigned
8 / 16 / 32 bit
Add and subtract

Adding without leaving the range

SHADD16 and SHADD8

Signed Halving Add 16 and Signed Halving Add 8.

Syntax

op{*cond*}{*Rd*,} *Rn*, *Rm*

where:

<i>op</i>	Is any of: SHADD16 Signed Halving Add 16 SHADD8 Signed Halving Add 8
<i>cond</i>	Is an optional condition code, see
<i>Rd</i>	Specifies the destination register.
<i>Rn</i>	Specifies the first operand register.
<i>Rm</i>	Specifies the second operand register.

Operation

Use these instructions to add 16-bit and 8-bit data and then to halve the result before writing the result to the destination register:

CM4 single cycle DSP instructions

OPERATION	INSTRUCTIONS	CM3	CM4
$16 \times 16 = 32$	SMULBB, SMULBT, SMULTB, SMULTT	n/a	1
$16 \times 16 + 32 = 32$	SMLABB, SMLABT, SMLATB, SMLATT	n/a	1
$16 \times 16 + 64 = 64$	SMLALBB, SMLALBT, SMLALTB, SMLALTT	n/a	1
$16 \times 32 = 32$	SMULWB, SMULWT	n/a	1
$(16 \times 32) + 32 = 32$	SMLAWB, SMLAWT	n/a	1
$(16 \times 16) \pm (16 \times 16) = 32$	SMUAD, SMUADX, SMUSD, SMUSDX	n/a	1
$(16 \times 16) \pm (16 \times 16) + 32 = 32$	SMLAD, SMLADX, SMLSD, SMLSDX	n/a	1
$(16 \times 16) \pm (16 \times 16) + 64 = 64$	SMLALD, SMLALDX, SMLSLD, SMLSLDX	n/a	1
$32 \times 32 = 32$	MUL	1	1
$32 \pm (32 \times 32) = 32$	MLA, MLS	2	1
$32 \times 32 = 64$	SMULL, UMULL	5-7	1
$(32 \times 32) + 64 = 64$	SMLAL, UMLAL	5-7	1
$(32 \times 32) + 32 + 32 = 64$	UMAAL	n/a	1
$32 \pm (32 \times 32) = 32$ (upper)	SMMLA, SMMLAR, SMMLS, SMMLSR	n/a	1
$(32 \times 32) = 32$ (upper)	SMMUL, SMMULR	n/a	1

Fractional numbers

Idea: Map signed or unsigned integer range into $\{-1 .. +1\}$

8 bit: $x = N / 256$ (unsigned) or $x = N / 128$ (signed)
16 bit: $x = N / 65536$ (unsigned) or $x = N / 32768$ (signed)

Example (16 bits signed fractional format “q15”):

0 $\rightarrow$ 0.0
0x7fff $\rightarrow$ +1 – epsilon (exactly $32767/32768 = 0.999695\dots$)
0x8000 $\rightarrow$ -1

- Multiplication is safe because the result can never leave the range.
- After the (integer) multiplication a shift needs to be done
- Additions and subtractions are dangerous as they can leave the range.
- Safe fractional computing can be done using saturating arithmetics.

Floating Point Support

Cortex M4 contains a FPU with support for

- Add
- Subtract
- Multiply
- Divide
- Multiply and accumulate
- Square root

The number format is single precision (32 bits)

Float number ranges

IEEE single precision (32 bit)

float

$\pm 10^{-37} \dots \pm 10^{+38}$

Mantissa: 23 bit = approx. 7 significant decimal digits

IEEE double precision (64 bit)

double

$\pm 10^{-307} \dots \pm 10^{+308}$

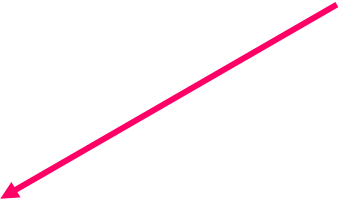
Mantissa: 52 bit = 15.65 significant decimal digits

(Distance Earth-Moon: 384401 km / $2^{52} = 0.085$ nm !)

Converting fix-point into float

$Z = 1101001001 . 100010010010101001010001$

$Z = 1 . 101001001100010010010101001010001 * 2^9$



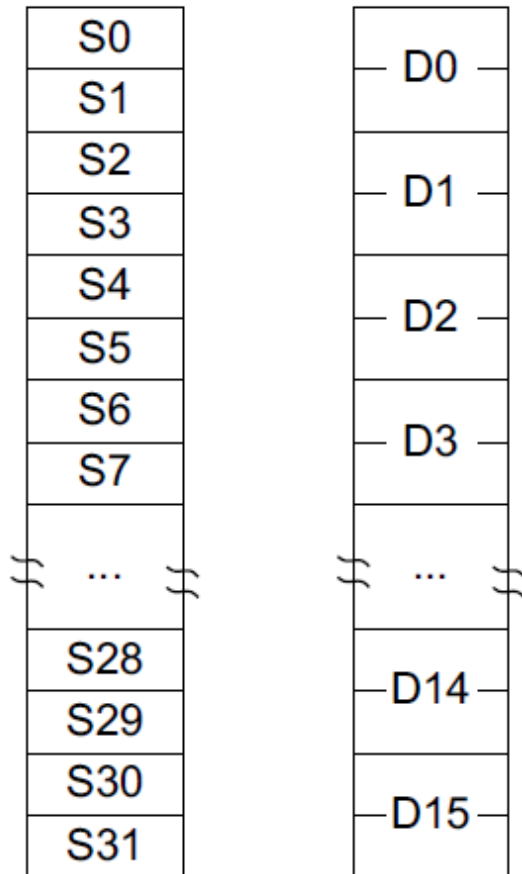
Mantissa

$= 1 + 0.101001001100010010010101001010001$

Exponent = 9

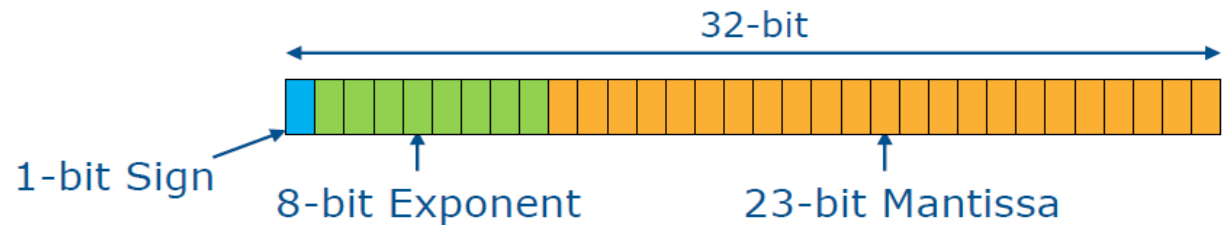
“Normalisation”

Floating Point Support

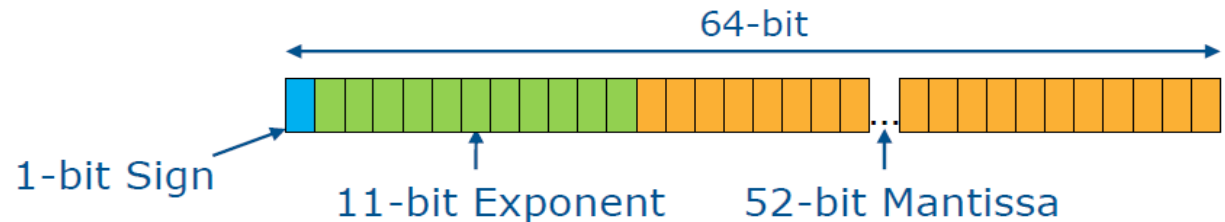


FPU register bank

Single precision : 32-bit coding



Double precision : 64-bit coding



Cortex M4F adds 32 16-bit float registers to the integer register bank.

Cortex-M4 FPU

VABS	VADD	VCMP	VCMPPE	VCVT	VCVTR	VDIV	VLDM
VLDR	VMLA	VMLS	VMOV	VMRS	VMSR	VMUL	VNEG
VNMLA	VMMLS	VNMUL	VPOP	VPUSH	VSQRT	VSTM	VSTR
VSUB	VFMA	VFMS	VFNMA	VFNMS			

PKH	QADD	QADD16	QADD8	QASX	QDADD	QDSUB	QSAX
QSUB	QSUB16	QSUB8	SADD16	SADD8	SASX	SEL	SHADD16
SHADD8	SHASX	SHSAX	SHSUB16	SHSUB8	SMLABB	SMLABT	SMLATB
SMLATT	SMLAD	SMLALBB	SMLALBT	SMLALTB	SMLALTT	SMLALD	SMLAWB
SMLAWT	SMLSD	SMLSLD	SMMLA	SMMLS	SMMUL	SMUAD	SMULBB

ADC	ADD	ADR	AND	ASR	B
CLZ	BFC	BFI	BIC	CDP	CLREX
CBNZ	CBZ	CMN	CMP	DBG	LDC
LDMIA	LDMDB	LDR	LDRB	LDRBT	LDRD
LDREX	LDREXB	LDREXH	LDRH	LDRHT	LDRSB
LDRSBT	LDRSHT	LDRSH	LDRT	MCR	LSL
LSR	MCRR	MLS	MLA	MOV	MOVT
MRC	MRRC	MUL	MVN	NOP	ORN
ORR	PLD	PLDW	PLI	POP	PUSH
RBIT	REV	REV16	REVSH	ROR	RRX

BKPT	BLX	ADC	ADD	ADR
BX	CPS	AND	ASR	B
DMB		BL		BIC
DSB	CMN	CMP		EOR
ISB	LDR	LDRB	LDM	
MRS	LDRH	LDRSB	LDRSH	
MSR	LSL	LSR	MOV	
NOP	REV	MUL	MVN	ORR
REV16	REVSH	POP	PUSH	ROR
SEV	SXTB	RSB	SBC	STM
SXTH	UXTB	STR	STRB	STRH
UXTH	WFE	SUB	SVC	TST
WFI	YIELD			

Cortex-M0/M0+/M1

RSB	SBC	SBFX
SDIV	SEV	SMLAL
SMULL	SSAT	STC
STMIA	STMDB	STR
STRB	STRBT	STRD
STREX	STREXB	STREXH
STRH	STRHT	STRT
SUB	SXTB	SXTH
TBB	TBH	TEQ
TST	UBFX	UDIV
UMLAL	UMULL	USAT
UXTB	UXTH	WFE
WFI	YIELD	IT

Cortex-M3

SMULBT	SMULTT
SMULTB	SMULWT
SMULWB	SMUSD
SSAT16	SSAX
SSUB16	SSUB8
SXTAB	SXTAB16
SXTAH	SXTB16
UADD16	UADD8
UASX	UHADD16
UHADD8	UHASX
UHSAX	UHSUB16
UHSUB8	UMAAL
UQADD16	UQADD8
UQASX	UQSAX
UQSUB16	UQSUB8
USAD8	USADA8
USAT16	USAX
USUB16	USUB8
UXTAB	UXTAB16
UXTAH	UXTB16

Cortex-M4

Floating point instruction set

Operation	Description	Assembler	Cycles
Absolute value	of float	VABS.F32	1
Addition	floating point	VADD.F32	1
Compare	float with register or zero	VCMP.F32	1
	float with register or zero	VCMP.E.F32	1
Convert	between integer, fixed-point, half-precision and float	VCVT.F32	1
Divide	Floating-point	VDIV.F32	14
Load	multiple doubles	VLDM.64	1+2*N, where N is the number of doubles.
	multiple floats	VLDM.32	1+N, where N is the number of floats.
	single double	VLDR.64	3
	single float	VLDR.32	2

Floating point instruction set cont.

Operation	Description	Assembler	Cycles
Pop	double registers from stack	VPOP.64	$1+2*N$, where N is the number of double registers.
	float registers from stack	VPOP.32	$1+N$ where N is the number of registers.
Push	double registers to stack	VPUSH.64	$1+2*N$, where N is the number of double registers.
	float registers to stack	VPUSH.32	$1+N$, where N is the number of registers.
Square-root	of float	VSQRT.F32	14
Store	multiple double registers	VSTM.64	$1+2*N$, where N is the number of doubles.
	multiple float registers	VSTM.32	$1+N$, where N is the number of floats.
	single double register	VSTR.64	3
	single float registers	VSTR.32	2
Subtract	float	VSUB.F32	1

Operation	Description	Assembler	Cycles
Move	top/bottom half of double to/from core register	VMOV	1
	immediate/float to float-register	VMOV	1
	two floats/one double to/from two core registers or one float to/from one core register	VMOV	2
	floating-point control/status to core register	VMRS	1
	core register to floating-point control/status	VMSR	1
Multiply	float	VMUL.F32	1
	then accumulate float	VMLA.F32	3
	then subtract float	VMLS.F32	3
	then accumulate then negate float	VNMLA.F32	3
	then subtract then negate float	VNMLS.F32	3
Multiply (fused)	then accumulate float	VFMA.F32	3
	then subtract float	VFMS.F32	3
	then accumulate then negate float	VFNMA.F32	3
	then subtract then negate float	VFNMS.F32	3
Negate	float	VNEG.F32	1
	and multiply float	VNMUL.F32	1

CM4F floating-point arithmetics

- Multiplication is (usually) safe.
- Addition / subtraction may lead to information / precision loss.
- Configurable handling of overflow / underflow situations.
- The FPU does not have user mode exception traps.
- Exception handling is tricky: Lazy stacking can be supported.

Summa Summarum

Computer architecture

Microprocessors / Micro-Controllers

8 bit ATMEL AVR

ARM Cortex M0

Ultra-low power designs

Embedded Application Architectures

ARM Cortex M3

Interrupt-processing

Memory-protection unit

ARM Cortex M4

DSP Instructions

Saturating arithmetics

Float arithmetics